

Key-Value Server

PRINCE
25M0809

Introduction :

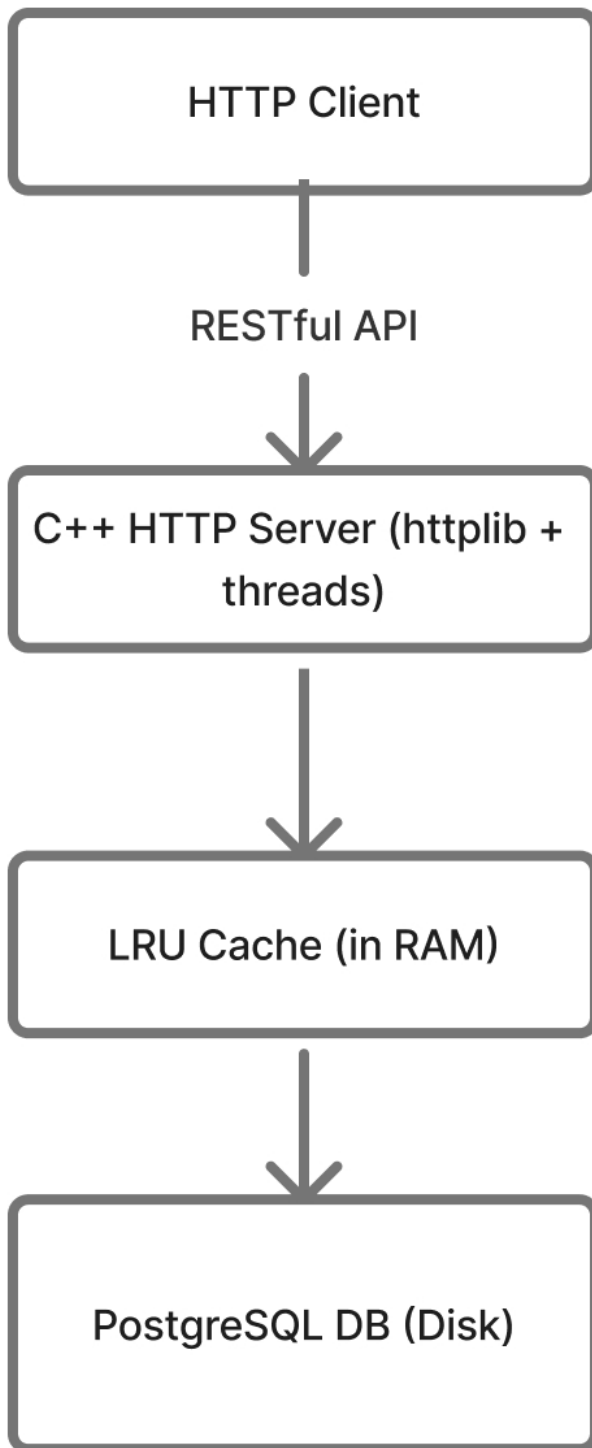
I have implemented a **RESTful** key-value store server in C++ using the [cpp-httplib](#) library .This server supports **create , read , delete** operations over http routes and stores data persistently in **PostgreSQL** database . An in-memory **LRU** based cache is also used to improve read performance . The server supports concurrency through multithreading and lock to ensure thread-safe access to shared data.

Also implemented a client to connect with server and receive commands from user to create , read and delete key-value in server .

Also implemented a multithreaded load generator that generates random requests .

System Design :

- The HTTP Server handles client requests via RESTful routes.
- The LRU cache stores recently used key-value pairs in memory.
- The PostgreSQL database persistently stores data in disk.
- Synchronization ensures safe concurrent access using locks.



Implementation Details :

Server →

- Implemented using cpp-httpplib library .
- 3 REST endpoints :
 - POST (/) - Create a new key-value pair .
 - GET (/:key) – Retrieve a value by key from server .
 - DELETE (/:key) – Delete a key-value pair .
- Each request is handled by a thread from threadpool implemented in cpp-httpplib.

Database →

- PostgreSQL using libpq library .
- Connection established using Pqconnectdb() .
- Used PqexecParams() to prevent SQL injection .

Cache →

- Implemented using doubly linked list and unordered map .
- Provides nearly $O(1)$ access to key-values stored in cache .

Client and Loadgenerator →

- User interactive client function to connect with server .
- Multi-threaded loadgenerator to generate random requests parallelly .

GITHUB Link →

Key – Value store

Files →

Server - server.cpp

cmd – g++ server.cpp -o server -lpq
./server #LRU cache capacity

Client – client.cpp

cmd – g++ client.cpp -o client
./client

Load generator – loadgeneraotr.cpp

cmd – g++ loadgeneraotr.cpp -o loadgeneraotr
./ loadgeneraotr